

Activity Recognition (ADL) Manual Run

Purpose

This procedure creates a native Linux expected result for the EgoVizML ADL-recognition pipeline and uses that result to test EgoModelKit with the same scientific preprocessing settings. The reference input is the approximately 17-second fridge video. EgoModelKit is then tested once with the original single video and once with the same session divided into two chronologically adjacent video files.

Required files

Place the following files in `~/Downloads`:

[EgoVizML.zip](#)
[EgoModelKit.zip](#)
[Detic.zip](#)
[detectron2.zip](#)
[fridge.mp4](#)
[fridge_part01.mp4](#)
[fridge_part02.mp4](#)

The split files must be chronological, must together represent the same fridge session, and should be approximately 10 seconds plus 7 seconds. Use the exact same three input files for all comparisons.

The following Shan reference items must already exist from the preceding Hand-Object Detector Manual Run guide:

- `~/hoc-reference/hand_object_detector` at commit `70146ca`
- Conda environment `handobj_new` with the compiled CPU detector extension
- `~/hoc-reference/hand_object_detector/models/res101_handobj_100K/pascal_voc/faster_rcnn_1_8_132028.pth`

1. Verify the resumed Linux account

Confirm the existing tools, NVIDIA driver, Miniconda installation, and Shan setup:

```
command -v git gcc g++ make wget unzip ffmpeg ffprobe  
nvidia-smi
```

```
source "$HOME/miniconda3/etc/profile.d/conda.sh"  
conda --version  
conda env list
```

```
test -f \  
"$HOME/hoc-reference/hand_object_detector/models/res101_handobj_100K/p  
ascal_voc/faster_rcnn_1_8_132028.pth"
```

The handobj_new environment and checkpoint test must succeed. A working NVIDIA driver is required for the native Detic stage. Shan remains CPU-only.

2. Extract and pin the source repositories

```
mkdir -p "$HOME/adl-reference"  
cd "$HOME/adl-reference"
```

```
unzip -q "$HOME/Downloads/EgoVizML.zip"  
unzip -q "$HOME/Downloads/EgoModelKit.zip"  
unzip -q "$HOME/Downloads/Detic.zip"  
unzip -q "$HOME/Downloads/detectron2.zip"
```

```
EV="$HOME/adl-reference/EgoVizML "  
EMK="$HOME/adl-reference/EgoModelKit"  
DETIK="$HOME/adl-reference/Detic"  
D2="$HOME/adl-reference/detectron2"  
HOC="$HOME/hoc-reference/hand_object_detector"
```

Create the clean EgoVizML reference branch:

```
cd "$EV"  
git reset --hard  
git clean -fdx
```

```
git checkout --detach \  
b3c24d065179289cd0d99091de06ba6fe54083c8
```

```
git switch -c adl-reference-paper
```

```
git config user.name "username"  
git config user.email "user@email.com"
```

```
git cherry-pick \  
c129075eef8f818947b250d1116d00267c4a9455
```

Verify the reference history:

```
git log -2 --oneline
```

```
test "$(git rev-parse HEAD^)" = \  
"b3c24d065179289cd0d99091de06ba6fe54083c8"
```

```
git diff --name-status HEAD^ HEAD
```

```
git merge-base --is-ancestor e5cc787 HEAD \  
&& echo "FAIL: line-ending commit is present" \  
|| echo "PASS: line-ending commit is excluded"
```

```
git merge-base --is-ancestor c3e4117 HEAD \  
&& echo "FAIL: instrumentation commit is present" \  
|| echo "PASS: instrumentation commit is excluded"
```

Expected committed diff: data/detic_class_mapping.csv is added and scripts/process_detic.py is modified. Both excluded-commit checks must print PASS.

Pin Detic, Detectron2, and the Detic submodules:

```
cd "$DETIK"  
git reset --hard  
git clean -fdx  
git checkout 436cda2a2347df60a7c66daca0e8c59f93dc5e79  
git submodule update --init --recursive  
cd "$D2"  
git reset --hard  
git clean -fdx  
git checkout e0ec4e189d438848521aee7926f9900e114229f5
```

```
cd "$EMK"
test "$(git rev-parse HEAD)" = \
    "5bf030f72b984a81a3e3cef949e87fe0088c9ced"
```

3. Apply the native Detic compatibility patch

```
"$HOME/miniconda3/bin/python" \
"$EMK/src/egomodelkit/resources/containers/adl_recognition/detic/patch
_egovizml_run_detic.py" \
"$EV/scripts/run_detic.py"
```

Restore CRLF line endings so the excluded normalization commit is not introduced incidentally:

```
"$HOME/miniconda3/bin/python" - "$EV/scripts/run_detic.py" <<'PY'
from pathlib import Path
import sys

path = Path(sys.argv[1])
data = path.read_bytes().replace(b"\r\n", b"\n").replace(b"\r", b"\n")
path.write_bytes(data.replace(b"\n", b"\r\n"))PY

cd "$EV"
git status --short
git diff --stat -- scripts/run_detic.py
git diff --numstat -- scripts/run_detic.py
```

Expected: `scripts/run_detic.py` is the only uncommitted file, with the limited API compatibility changes. Do not commit it.

4. Create the EgoVizML core environment

```
source "$HOME/miniconda3/etc/profile.d/conda.sh"

conda create -y \
    --name adl_ref \
    python=3.11
```

```
conda activate adl_ref
python -m pip install --upgrade pip setuptools wheel
```

```
python -m pip install \
    torch==2.11.0 \
    torchvision==0.26.0 \
    --index-url https://download.pytorch.org/whl/cpu
```

```
python -m pip install \
    moviepy==1.0.3 \
    opencv-python==4.9.0.80 \
    pandas==2.1.4 \
    polars==0.19.19 \
    scikit-learn==1.3.1 \
    joblib==1.3.2 \
    tqdm==4.66.1 \
    numpy==1.26.4 \
    pyarrow==18.0.0
```

```
cd "$EV"
python -m pip install --no-deps -e .
```

Standard OpenCV is intentional because unmodified EgoVizML calls `cv2.destroyAllWindows()`.
Confirm the inference-only runtime and classifier:

```
python - <<'PY'
import cv2
import joblib
import moviepy
import numpy
import pandas
import polars
import pyarrow
import sklearn
import torch
import torchvision
import tqdm

from egoviz.cdss_utils.video_processing import
process_videos_in_folder
```

```

from egoviz.models import inference
from egoviz.models.processing import load_pickle

model = inference.load_production_model(
    "models/binary_active_logreg.joblib"
)

cv2.destroyAllWindows()

print("EgoVizML reference runtime imports: PASS")
print("OpenCV:", cv2.__version__)
print("NumPy:", numpy.__version__)
print("Pandas:", pandas.__version__)
print("Polars:", polars.__version__)
print("PyArrow:", pyarrow.__version__)
print("scikit-learn:", sklearn.__version__)
print("PyTorch:", torch.__version__)
print("TorchVision:", torchvision.__version__)
print("CUDA available in core environment:",
torch.cuda.is_available())
print("Classifier feature count:", len(model.feature_names))
print("OpenCV GUI cleanup: PASS")
PY

```

5. Create the native Detic GPU environment

```

source "$HOME/miniconda3/etc/profile.d/conda.sh"
conda deactivate 2>/dev/null || true
conda env remove -y --name adl_detic

conda create -y \
    --name adl_detic \
    python=3.10

conda activate adl_detic

conda install -y \
    -c conda-forge \

```

```

gcc_linux-64=13 \
gxx_linux-64=13

conda install -y \
  -c nvidia \
  cuda-toolkit=12.8

python -m pip install --upgrade pip setuptools wheel
python -m pip install ninja gdown==5.2.0

python -m pip install \
  torch==2.11.0 \
  torchvision==0.26.0 \
  torchaudio==2.11.0 \
  --index-url https://download.pytorch.org/whl/cu128

python -m pip install \
  numpy==1.26.4 \
  Pillow==9.5.0

```

Configure the compiler and CUDA paths:

```

export CC="$CONDA_PREFIX/bin/x86_64-conda-linux-gnu-gcc"
export CXX="$CONDA_PREFIX/bin/x86_64-conda-linux-gnu-c++"
export CUDAHOSTCXX="$CXX"
export CUDA_HOME="$CONDA_PREFIX"
export FORCE_CUDA=1
export TORCH_CUDA_ARCH_LIST="7.5;8.0;8.6;8.9;9.0;12.0+PTX"
export
LD_LIBRARY_PATH="$CONDA_PREFIX/lib:$CONDA_PREFIX/lib64:${LD_LIBRARY_PATH:-}"

"$CC" --version
"$CXX" --version
command -v nvcc
nvcc --version

```

Both compiler commands must report version 13.x, not 14.x. Clean and compile Detectron2:

```
cd "$D2"
rm -rf build detectron2.egg-info
find detectron2 -maxdepth 1 -type f -name '_C*.so' -delete
```

```
MAX_JOBS=2 \
FORCE_CUDA=1 \
CC="$CC" \
CXX="$CXX" \
CUDAHOSTCXX="$CUDAHOSTCXX" \
python -m pip install \
    --no-build-isolation \
    --verbose \
    -e .
```

Install Detic requirements, then restore the compatible OpenCV/NumPy/Pillow set:

```
cd "$DETIIC"
python -m pip install -r requirements.txt
```

```
python -m pip install \
    --force-reinstall \
    numpy==1.26.4 \
    Pillow==9.5.0 \
    opencv-python==4.9.0.80
```

Confirm CenterNet2 and all compiled imports:

```
cd "$DETIIC"
git submodule update --init --recursive
```

```
test -f \
    "$DETIIC/third_party/CenterNet2/centernet/config.py" \
    && echo "CenterNet2: PASS"
```

```
export CUDA_HOME="$CONDA_PREFIX"
export
PYTHONPATH="$DETIIC/third_party/CenterNet2:$D2:$DETIIC:$EV:${PYTHONPATH:
-}"
```

```
python - <<'PY'
import cv2
```



```

import numpy
import PIL
import torch
import torchvision

from centernet.config import add_centernet_config
from detectron2 import _C
from detectron2.config import get_cfg
from detectron2.layers import nms
from detic.config import add_detic_config

cfg = get_cfg()
add_centernet_config(cfg)
add_detic_config(cfg)

print("Detic environment imports: PASS")
print("Detectron2 CUDA extension: PASS")
print("CenterNet2 config: PASS")
print("Detic config: PASS")
print("OpenCV:", cv2.__version__)
print("NumPy:", numpy.__version__)
print("Pillow:", PIL.__version__)
print("PyTorch:", torch.__version__)
print("PyTorch CUDA:", torch.version.cuda)
print("TorchVision:", torchvision.__version__)
print("CUDA available:", torch.cuda.is_available())

if torch.cuda.is_available():
    print("GPU:", torch.cuda.get_device_name(0))
PY

```

The `timm.models.layers` deprecation warning is informational. Native Detectron2 compilation requires `nvcc`; the machine also needs a compatible NVIDIA driver.

6. Download the pinned Detic weights

```

source "$HOME/miniconda3/etc/profile.d/conda.sh"
conda activate adl_detic

```

```
cd "$DETIK"
mkdir -p models

python -m gdown --fuzzy \
  "https://drive.google.com/file/d/1WFgtv1_o30BzNICmSJmA-a-92ZIHcMS4/view?usp=sharing" \
  -O models/Detic_LCOCO121k_CLIP_SwinB_896b32_4x_ft4x_max-size.pth

test -s \
  models/Detic_LCOCO121k_CLIP_SwinB_896b32_4x_ft4x_max-size.pth

sha256sum \
  models/Detic_LCOCO121k_CLIP_SwinB_896b32_4x_ft4x_max-size.pth
```

7. Prepare the native reference and the two EgoModelKit inputs

Prepare the native reference workspace from the original unsplit fridge video:

```
RUN_ROOT="$HOME/adl-reference-run-paper"
DATA_ROOT="$RUN_ROOT/egoviz_data"
ADL_DIR="$DATA_ROOT/meal-preparation-cleanup"
FRAME_DIR="$RUN_ROOT/model_frames"
RESULTS_DIR="$RUN_ROOT/results"

rm -rf "$RUN_ROOT"
mkdir -p "$ADL_DIR" "$FRAME_DIR" "$RESULTS_DIR"

cp "$HOME/Downloads/fridge.mp4" \
  "$ADL_DIR/video001.MP4"

cat > "$RUN_ROOT/input_mapping.csv" <<'CSV'
staged_video,input_video,session_id,session_sort_index
video001.MP4,fridge.mp4,session001,1
CSV
```

Prepare separate folders for the packaged single-video and split-session tests:

```
TEST_INPUT_ROOT="$HOME/adl-paper-test-inputs"
SINGLE_INPUT="$TEST_INPUT_ROOT/fridge-single"
SPLIT_INPUT="$TEST_INPUT_ROOT/fridge-split-session"
```

```

rm -rf "$TEST_INPUT_ROOT"
mkdir -p "$SINGLE_INPUT" "$SPLIT_INPUT"

cp "$HOME/Downloads/fridge.mp4" \
  "$SINGLE_INPUT/fridge.mp4"

cp "$HOME/Downloads/fridge_part01.mp4" \
  "$SPLIT_INPUT/fridge_part01.mp4"
cp "$HOME/Downloads/fridge_part02.mp4" \
  "$SPLIT_INPUT/fridge_part02.mp4"

for video in \
  "$SINGLE_INPUT/fridge.mp4" \
  "$SPLIT_INPUT/fridge_part01.mp4" \
  "$SPLIT_INPUT/fridge_part02.mp4"
do
  printf '%s: ' "$(basename "$video")"

  env -u LD_LIBRARY_PATH \
    /usr/bin/ffprobe \
    -v error \
    -show_entries format=duration \
    -of default=noprint_wrappers=1:nokey=1 \
    "$video"
done

```

The split folder must sort part01 before part02. The combined duration should approximately equal the original fridge duration.

8. Extract the paper-settings native comparison frames

```

source "$HOME/miniconda3/etc/profile.d/conda.sh"
conda activate adl_ref

EV="$HOME/adl-reference/EgoVizML"
RUN_ROOT="$HOME/adl-reference-run-paper"
DATA_ROOT="$RUN_ROOT/egoviz_data"

```

```
ADL_DIR="$DATA_ROOT/meal-preparation-cleanup"
FRAME_DIR="$RUN_ROOT/model_frames"

rm -rf "$ADL_DIR/subclips" "$FRAME_DIR"
mkdir -p "$FRAME_DIR"

cd "$EV"
python scripts/video_to_subclips_and_frames.py \
    "$ADL_DIR" \
    --subclip_length 60 \
    --fps 10 \
    --frame_fps 1 \
    2>&1 | tee "$RUN_ROOT/subclip-extraction.log"
```

Confirm one subclip and one frame directory:

```
find "$ADL_DIR/subclips" \
    -maxdepth 1 \
    -type f \
    -name '*.MP4' \
    -printf '%f\n' \
    | sort

find "$ADL_DIR/subclips" \
    -mindepth 1 \
    -maxdepth 1 \
    -type d \
    -printf '%f\n' \
    | sort

find "$ADL_DIR/subclips/video001--1" \
    -maxdepth 1 \
    -type f \
    -name 'frame_*.jpg' \
    | wc -l
```

Expected: `video001--1.MP4`, directory `video001--1`, and 60 JPEG frames. MoviePy end-of-source warnings are expected because the short fixture is padded to one minute.

Flatten the frames into one uniquely named directory for Detic and Shan:

```

python - "$ADL_DIR/subclips" "$FRAME_DIR" <<'PY'
from pathlib import Path
import shutil
import sys

subclips = Path(sys.argv[1])
output = Path(sys.argv[2])
output.mkdir(parents=True, exist_ok=True)

for old in output.glob("*.jpg"):
    old.unlink()

count = 0
for clip_dir in sorted(path for path in subclips.iterdir() if
path.is_dir()):
    for frame in sorted(clip_dir.glob("frame_*.jpg")):
        frame_token = frame.stem.replace("frame_", "frame", 1)
        target = output / f"{clip_dir.name}_{frame_token}.jpg"
        shutil.copy2(frame, target)
        count += 1

print("Comparison frames:", count)
PY

find "$FRAME_DIR" -maxdepth 1 -type f -name '*.jpg' | wc -l

```

Expected comparison frame count: 60.

9. Run Detic and Shan on the same 60 frames

Run Detic:

```

source "$HOME/miniconda3/etc/profile.d/conda.sh"
conda activate adl_detic

EV="$HOME/adl-reference/EgoVizML"
DETIK="$HOME/adl-reference/Detic"
D2="$HOME/adl-reference/detectron2"
RUN_ROOT="$HOME/adl-reference-run-paper"

```

```

ADL_DIR="$RUN_ROOT/egoviz_data/meal-preparation-cleanup"
FRAME_DIR="$RUN_ROOT/model_frames"

export CUDA_HOME="$CONDA_PREFIX"
export
PYTHONPATH="$DETIK/third_party/CenterNet2:$D2:$DETIK:$EV:${PYTHONPATH:
-}"
export CUDA_VISIBLE_DEVICES=0

rm -rf "$ADL_DIR/detic"
mkdir -p "$ADL_DIR/detic"

cd "$DETIK"
python "$EV/scripts/run_detic.py" \
  --input-dir "$FRAME_DIR" \
  --output-dir "$ADL_DIR/detic" \
  --no-images \
  --confidence-threshold 0.3 \
  --num-workers 1

find "$ADL_DIR/detic" \
  -maxdepth 1 \
  -type f \
  -name '*_detic.pkl' \
  | wc -l

```

Expected Detic pickle count: 60.

Run the previously prepared hand-object detector on the exact same flat frame files:

```

source "$HOME/miniconda3/etc/profile.d/conda.sh"
conda activate handobj_new

HOC="$HOME/hoc-reference/hand_object_detector"
RUN_ROOT="$HOME/adl-reference-run-paper"
ADL_DIR="$RUN_ROOT/egoviz_data/meal-preparation-cleanup"
FRAME_DIR="$RUN_ROOT/model_frames"

rm -rf "$ADL_DIR/shan"
mkdir -p "$ADL_DIR/shan"

```

```

cd "$HOC"
python demo.py \
  --dataset pascal_voc \
  --cfg cfgs/res101.yml \
  --net res101 \
  --load_dir models \
  --image_dir "$FRAME_DIR" \
  --save_dir "$ADL_DIR/shan" \
  --checksession 1 \
  --checkepoch 8 \
  --checkpoint 132028 \
  --thresh_hand 0.5 \
  --thresh_obj 0.5

find "$ADL_DIR/shan" \
  -maxdepth 1 \
  -type f \
  -name '*_shan.pkl' \
  | wc -l

```

Expected Shan pickle count: 60. Do not add --cuda.

Confirm every frame has both outputs using Python 3.8-compatible suffix removal:

```

python - "$ADL_DIR/detic" "$ADL_DIR/shan" <<'PY'
from pathlib import Path
import sys

detic_dir = Path(sys.argv[1])
shan_dir = Path(sys.argv[2])

detic_suffix = "_detic.pkl"
shan_suffix = "_shan.pkl"

detic = {
    p.name[:-len(detic_suffix)]
    for p in detic_dir.glob(f"*{detic_suffix}")
}

```

```

shan = {
    p.name[:-len(shan_suffix)]
    for p in shan_dir.glob(f"*{shan_suffix}")
}

if detic != shan:
    raise RuntimeError(
        f"Unpaired outputs: Detic-only={len(detic - shan)}, "
        f"Shan-only={len(shan - detic)}"
    )

print("Paired Detic/Shan frames:", len(detic))
PY

```

Expected: Paired Detic/Shan frames: 60.

10. Create the native expected ADL outputs

```

source "$HOME/miniconda3/etc/profile.d/conda.sh"
conda activate adl_ref

EV="$HOME/adl-reference/EgoVizML"
RUN_ROOT="$HOME/adl-reference-run-paper"
DATA_ROOT="$RUN_ROOT/egoviz_data"
RESULTS_DIR="$RUN_ROOT/results"

for name in \
    communication-management \
    functional-mobility \
    grooming-health-management \
    home-management \
    leisure-other-activities \
    meal-preparation-cleanup \
    self-feeding
do
    mkdir -p "$DATA_ROOT/$name/detic" "$DATA_ROOT/$name/shan"
done

```



```

cd "$EV"
python scripts/process_all_preds.py \
    "$DATA_ROOT" \
    --active_iou 0.8

cp "$DATA_ROOT/all_preds.pkl" \
    "$RESULTS_DIR/all_preds.pkl"

```

Create the stable CSV export wrapper. It calls the original EgoVizML feature and classifier functions and fills absent trained features with zero:

```

cat > "$RUN_ROOT/predict_adl_reference.py" <<'PY'
from pathlib import Path
import sys
import polars as pl
from egoviz.models import inference, processing

all_preds_path = Path(sys.argv[1])
output_dir = Path(sys.argv[2])
output_dir.mkdir(parents=True, exist_ok=True)

preds = processing.load_pickle(str(all_preds_path))
frame_df = processing.generate_df_from_preds(preds)
features = processing.generate_binary_presence_df(frame_df)
scaled = processing.row_wise_min_max_scaling(features).fillna(0.0)

model = inference.load_production_model(
    "models/binary_active_logreg.joblib"
)

for column in model.feature_names:
    if column not in scaled.columns:
        scaled[column] = 0.0

results = inference.predict(
    pl.from_pandas(scaled), model
).to_pandas().sort_values("video")

probability_columns = [
    column for column in results.columns

```

```

        if column.startswith("prob_")
    ]

    results["predicted_label_readable"] = (
        results[probability_columns]
        .idxmax(axis=1)
        .str.replace("prob_", "", regex=False)
    )

    results["predicted_probability"] =
    results[probability_columns].max(axis=1)

    for column in results.columns:
        if results[column].dtype == "object":
            results[column] = results[column].map(
                lambda value: repr(value) if isinstance(value, bytes) else
                str(value)
            )

    results.to_csv(output_dir / "adl_predictions.csv", index=False)

    summary_columns = [
        "video",
        "adl",
        "predicted_label_readable",
        "predicted_probability",
        "predicted_class",
        *probability_columns,
    ]
    results[summary_columns].to_csv(
        output_dir / "adl_predictions_summary.csv",
        index=False,
    )

    print(output_dir / "adl_predictions.csv")
    print(output_dir / "adl_predictions_summary.csv")
PY

cd "$EV"
python "$RUN_ROOT/predict_adl_reference.py" \

```

```
"$RESULTS_DIR/all_preds.pkl" \  
"$RESULTS_DIR"
```

11. Confirm and package the expected result

```
RUN_ROOT="$HOME/adl-reference-run-paper"  
RESULTS_DIR="$RUN_ROOT/results"  
  
find "$RESULTS_DIR" \  
  -maxdepth 1 \  
  -type f \  
  -printf '%f\n' \  
  | sort  
  
python - \  
  "$RESULTS_DIR/all_preds.pkl" \  
  "$RESULTS_DIR/adl_predictions_summary.csv" <<'PY'  
from pathlib import Path  
import pickle  
import sys  
import pandas as pd  
  
all_preds_path = Path(sys.argv[1])  
summary_path = Path(sys.argv[2])  
  
with all_preds_path.open("rb") as file:  
    all_preds = pickle.load(file)  
  
summary = pd.read_csv(summary_path)  
  
print("all_preds frame keys:", len(all_preds))  
print("prediction rows:", len(summary))  
print(summary.to_string(index=False))  
  
if len(all_preds) != 60:  
    raise RuntimeError(f"Expected 60 frame keys, found  
{len(all_preds)}")
```

```
if len(summary) != 1:
    raise RuntimeError(f"Expected 1 prediction row, found
{len(summary)}")
PY
```

```
cat > "$RUN_ROOT/reference_parameters.txt" <<'TXT'
logical_subclip_seconds=60
temporary_subclip_fps=10
model_frame_fps=1
additional_resize=none
active_iou_argument=0.8
detic_confidence_threshold=0.3
shan_hand_threshold=0.5
shan_object_threshold=0.5
TXT
```

```
cd "$RUN_ROOT"
zip -r \
```

```
"$HOME/Downloads/adl-recognition-paper-settings-linux-expected-results
.zip" \
    results \
    input_mapping.csv \
    reference_parameters.txt
```

Expected primary files: `all_preds.pkl`, `adl_predictions.csv`, and `adl_predictions_summary.csv`. The summary has one row.